ICT374 Lab 4:
Job Control, Dealing with File Systems Using `du`, `df`, `mount` and `umount`; Programming with `struct` and Function Pointer; Working with Process Related System Calls, Creating Process Hierarchy; Obtaining Process Environment, Loading and Executing Program in Process, and Creating Daemon Process.

Learning Objectives:

1. Learn to use shell's pipeline facility
2. Learn to run programs in the background
3. Learn to use command `du` to find disk usage
4. Learn to use command `df` to check mounted file systems
5. Learn to use commands `mount` and `umount` to mount and unmount file systems
6. Learn to use command `shutdown` to shutdown your operating system
7. Understand and be able to use C's `struct`
8. Understand and be able to use function pointers
9. Understand and be able to create new processes with the `fork` system call.
10. Understand and be able to use `exec` functions to run a new program inside an existing process.
11. Understand and be able to synchronize with child processes using `wait` and `waitpid` system calls.
12. Understand and be able to use process related system calls or functions such as `sleep`, `exit`, `getpid`, `getppid`, `getgid`, `setgid` and `setsid`.
13. Understand and be able to obtain process environment.
14. Understand and be able to create and use daemon processes.

Required Reading:

- Lecture Notes for Topic 4
- Rute: Ch 19 and Ch 22
- A mini manual for vi editor
- The manual pages of the process related system calls and functions

Unix Exercises

The following exercises are designed to get you familiar with Unix (Linux) systems. More information about these basic Unix commands is available from the relevant chapters in Rute. See the Required Readings.

1. **Shell pipelines**

   In most shells, you can create a pipe and link the standard output from one process with the standard input from the second process. In this way the output of one process feeds the second process. The following command line lists all HTTP processes:

```
% ps -el | grep http
```

In the above example, the output from `ps` is used as the input of the `grep` process. The following command line lets you to type multiple lines from your keyboard and then sort these lines and save the lines into the file `foo`:

```
% cat | sort > foo
```

Note you finish the input by typing CTRL-D.

Try the above command lines.

2. **Execute process in the background**

   Most Unix shells support job control. A job is simply a pipeline of processes, such as these:

   ```
   ps -el | more
   cat | cat | cat | cat
   ```

   With job control, you can run several jobs simultaneously from the same terminal. However, at any given time, only one of these jobs can be in the *foreground*. All the other jobs are said to in the *background*. Only the foreground job can read terminal input, but all jobs can send their outputs to the terminal screen.

   To run a job in the background, we use &, for example:

   ```
   % sleep 100 &
   ```

   By running command `sleep 100` in the background, the shell allows us to issue the next command immediately, not to wait until the `sleep` has finished.

   You can use command `jobs` to see the jobs that are currently running in the background. You can use command `fg` to bring a job from the background to the foreground.

   Run the `sleep 100` command in the background and then bring it to the foreground.

3. **Check disk usage by files and directories**

   The command `du` can be used to find the disk space used by a file or a directory. However, its most useful function is to find out the total disk usage by a directory and all files and subdirectories under that directory. The following command displays the grant total of disk usage by the directory `/usr/bin`:

   ```
   % du --max-depth=1 -h /usr/bin
   ```

   Try the above command on different directories such as your home directory, `/tmp`, `/var`, etc.

4. **Display the information about filesystems**

   In Unix, a filesystem is the file data structure created on a disk partition. In a filesystem, the files are organised logically in a hierachical manner, much like a upside-down tree. In fact all filesystems are connected into a single tree. Use `df` to see the disk usage of each filesystem, its mount point, and the device name of each partition. For example, the

following command on eris displays information about two filesystems created on disk partitions::

```
S900432D@eris:~$ df
Filesystem           1K-blocks      Used Available Use% Mounted on
/dev/sda1              9614116   2208336   6917408  25% /
tmpfs                  1037500         4   1037496   1% /dev/shm
/dev/sda6             65361992    436856  61604904   1% /home
S900432D@eris:~$
```

The root filesystem is created on a disk partition whose device name is `/dev/sda1`. The other disk filesystem is created on a disk partition whose device name is `/dev/sda6` and is mounted on directory `/home`. The root filesystem is 25% used and the other is 1% used.

Try this command on your machine.

5. **Mount and unmount filesystems**

   In Unix, to access the files stored in a filesystem (fs), the fs must not only be physically present on the machine, it must also be *mounted*. On the other hand, if you want to reformat the disk partition or to create a new filesystem format (say from ext3 to vfat), you need to unmount the fs first. The commands involved are `mount` and `umount`. Usually you need to have root privilledge to perform this type of operations (the exact details depend on how the system is configured).

   We will not practice how to mount or unmount filesystems here as you may need root priviledge to do so. However any user can check the status of mounted filesystems using `mount` command:

```
S900432D@eris:~$ mount
/dev/sda1 on / type ext3 (rw,errors=remount-ro)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
usbfs on /proc/bus/usb type usbfs (rw)
tmpfs on /dev/shm type tmpfs (rw)
devpts on /dev/pts type devpts (rw,gid=5,mode=620)
/dev/sda6 on /home type ext3 (rw)
nfsd on /proc/fs/nfsd type nfsd (rw)
S900432D@eris:~$ mount -t msdos /dev/sda1 /tmp
mount: only root can do that
S900432D@eris:~$
```

6. **Shutting down Unix properly**

   All modern operating systems, including all Unix-like systems as well as newer MicroSoft Windows, use disk cache extensively to improve efficiency of disk access. Therefore, to turn off a computer, you must first shut down your OS to allow it to flush the memory buffers into the physical disks. Otherwise, you may damage the your files and file systems. The command to shut down Unix is `/sbin/shutdown`. However you need to login as root to use this command.

C Programming Exercises

Note, all exercises must be done via the terminal commands, not GUI tools. Do not use IDE to build your programs. Do not use GUI file managers to manage your directories and files. Also,

each exercise should be in a separate directory, which itself, is under a directory called `lab04`

1. **Create Structures with `struct` tags**

   In C, you can create a collection of data with structures. Unlike arrays where all elements are homogeneous (of same type), the data in a structure can have different types and different sizes. Look at the following example

   ```
   struct Student {
        char *name;              // point to a name string
        int id;                  // student number
        char enroll;             // enrollment option: D or X
   };

   struct Student john;

   ...

   john.name = "John Smith";
   john.id = 12345678;
   john.enroll = 'D';
   ```

   We first declare a structure tag called `struct Student` which contains three pieces of data: a char pointer to point to the name of the student, an integer for the student number, and a char for the enrollment option.

   A structure tag is pretty much like a type (or a class name if you are familiar with Java or C++). You can use it to declare a new structure variable. In the above example, we use the structure tag `struct Student` to declare a variable `john` to store the information about student john. The next three statements assign values to the three structure fields of `john`. You use operator "." to access structure fields, as in `john.name`.

   Write a C program to read student information from the user's terminal and assign the information to a structure variable such as `john` as the above. Then print out the values of the variable.

2. **Pass structures to functions**

   How would you pass a structure to a function? Look at the following example:

   ```
   void printStudent(struct Student s)
   {
       printf("Student name: %s\n", s.name);
       printf("Student number: %d\n", s.id);
       printf("Student enrollment option: %c\n", s.enroll);
       return;
   }
   ```

   To call the above function, we do

   ```
   printStudent(john);
   ```

   Question: what exactly is passed to function `printStudent` when we call it with `printStudent(john)`? Is it a *copy* of structure `john` or is it just the address of the structure `john`?

   Answer the above question. Then write a program to verify your answer.

3. **Change a structure from inside of a function**

   If you have completed exercise 2 above, you probably will have found out that what is passed to the function is a *copy* of the structure `john`. Therefore the formal parameter `s` inside function `printStudent` actually represents the local copy of the structure. Any change made to that local copy would not affect the structure `john` in any way.

   However on occasions we need to change the structure argument inside a function. For example, we want to input student details from inside function. In this case, we should pass the address of the structure to the function. The following example illustrates how this is achieved:

```
void getStudent(struct Student *s)
{
    printf("Type the name of the student: ");
    s->name = malloc(100);   // assume name has less than 100 letters
    fgets(s->name, 100, stdin);

    printf("\nType the student numner: ");
    scanf("%d", &(s->id));

    printf("\nType the student enrollment option (D or X): ");
    scanf("%c", &(s->enroll)); // watch out for a bug here!

    return;
}
```

   To get input into the structure `john`, we call the function:

```
getStudent(&john);
```

   Note that the formal parameter `s` in function `getStudent` is a pointer to structure. To access a structure field through the pointer to that structure, you should use operation `->`, such as in

```
    s -> name
```

   Also, you need to pass the address of the structure (ie `&john`) to the function as in

```
getStudent(&john);
```

   Write a program that includes one input function and one output function. The input function is similar to `getStudent` that reads student details from the user into a student structure. The output function is similar to `printStudent` that prints out the details in a student structure. Your main function should then call the input function to read student details from the user into a student structure variable and then print out the details in that structure by calling the output function.

4. **Function pointers**

   In C, you can declare a *variable* that points to a function. You can then use the variable to make the function call. The following example declares a function pointer variable, `fp`, that points to an integer function (that takes an integer argument):

```
int (*fp)(int);
```

The following code defines an integer function that takes an integer argument:

```
int square(int n)
{
    return n*n;
}
```

We can assign the address of function `square` to variable `fp`:

```
fp = square;
```

Now we can call the function `square` either directly as in

```
n = square(10);  // should return 100
```

or indirectly via the function pointer `fp`:

```
n = fp(10);    // should also return 100
```

Write a program to print out the address of its function. Assign the function to a function pointer variable and then test that function pointer variable by calling the function using the function pointer.

Unix Programming Exercises

1. **Create a child process**

   Write a small program that creates one child process.

2. **Distinguish between parent and child processes**

   Improve the program in Exercise 1 above: this time the parent should print out "I am the parent" and the child should print out "I am the child". In addition, each process should print its own pid (man getpid).

3. **Wait for child**

   Improve the program in Exercise 2 above: this time the parent waits until the child terminates and then prints "the child has terminated".

4. **Check the pid and ppid**

   Improve the program in Exercise 3 above: this time the child sleeps for 100 seconds before it terminates (`man -S 3 sleep`). Run your program in the background (use &). Use that `ps` command to see both processes (ps -lH). Check each process's pid and ppid to see if they match what have been printed by your program.

5. **Get the exit status of child process**

   Change the program in Exercise 4 so that the child terminates with an exit status (such as 10). Obtain this exit status from the parent process and print it out.

6. **How many child processes are created?**

   Try to work out the answer of this question mentally: what would happen if you call fork

twice in the following way:

```
fork();
fork();
```

How many child processes would be created? What are the relationships between these processes? Complete the following program to verify your answer:

```
.....
fork();
fork();
print my pid and my ppid (ie parent pid)
sleep(1000);  // so that the process does not quit too quickly
.....
```

While running the above program in one terminal, display the information about these processes in another terminal. What `ps` options would you need to show these processes and their parent-child relationship?

Hint: check Unix Exercise 6 in Lab 3. One possible command is

```
%  ps -efH | grep your_username
```

7. **Creating multiple direct child processes**

   Write a program that creates *n* direct child processes. For *i*th child, it prints "Child i" and its pid and ppid numbers. It then sleeps for *i* minutes before termination.

   In creating the child processes you must make sure that all children are the *direct* children of the original process. Use command `ps` to verify that this is indeed the case.

   Hint: consider the following program structure:

```
for (i=1; i<=n; ++i) {
    pid = fork();
    if (pid==0) {  // child
        print Child i:  my pid and my ppid
        sleep (i*60);   // sleep for i minutes
        exit(0);
    }
}
```

   Make sure you genuinely understand how the above code works.

   Test your program with different *n* values (e.g., 3, 4, 5) and verify the parent-child relationship with the program outputs as well as the output of the `ps` command.

8. **Loading a program in a process to execute it**

   Write a program that executes the command "ls -lt". Please do not use the function "system".

9. **Would a call to an `exec` function return?**

   In what circumstances would an exec call return? What output would you expect to see for the following code?

```
execlp("ls", "ls", "-l", NULL);
execlp("date", "date", NULL);
```

Write down your answer to the question first. Then write a program to verify your answer.

10. **Obtaining process environment**

    Write a program that 1) prints a list of its environment variables (and their values); 2) print out the current directory of the process as well as the user who invokes the program.

    Hints: 1) read Section 7 of Lecture Notes for Topic 4; 2) use the environment variables PWD and USER.

11. **Creating child process to execute a command**

    Write a program that keeps reading one string at a time from the keyboard. Each time a string is read, it is treated as a command name, and is executed in a child process. The program stops once the user types "bye". Test your program by typing the following strings: ls, ps, date, bye.

    You may adopt the following program structure:

```
char command[COMMAND_LINE_SIZE];
pid_t pid;

while (1) {
    display a prompt such as "%" or "$"
    read a line from user and store it in array command
    if command is "bye"
    break;
    if ((pid = fork()) <  0) {
        perror("fork");
        exit(1);
    }
    if (pid==0) { // child
        load and execute command
    }
    wait for the child to finish
}
```

12. **Creating a daemon process**

    Write a program that turns itself (actually its child) into a daemon. The daemon process simply writes one line of log "I have nothing to do" to a log file once every 10 seconds. Your program should print the daemon's pid at the beginning.

    Check that your daemon process is running using `ps` command (what options are needed?). Compare the pid from the `ps` command with that from your program. Check whether your daemon has a control terminal.

    Make sure you kill the daemon process using `kill` command at the end of this exercise.

    Hints: read Section 10 of Topic 4. You may use `daemon_init` function from that section.

The Weekly Lab

Submit the following exercises:

1. C Programming Exercise 4.
2. Unix Programming Exercise 7.
3. Unix Programming Exercise 11.
4. Unix Programming Exercise 12.

This weekly lab must be submitted to the LMS. The deadline of submission is given in the Unit Information Page of the LMS. You must adhere to the instructions in the document titled The Requirements for Documenting and Submitting Weekly Labs when preparing and submitting a weekly lab.

Dr Hong Xie
Unit Coordinator

Last update: Thursday, 21 August 2025

Document author: Dr. H. Xie, Unit Coordinator, ICT374 Operating Systems and Systems Programming
Last Modified: Thursday, 21-Aug-2025 21:41:08 AWST
Disclaimer & Copyright Notice © 2026 Murdoch University
This document is relevant for 2025 only